# Studio Nx

Backend(FastAPI/Python) 01 — error_json() · app_error_handler()

2026-10-06

# Backend(FastAPI/Python) 01 — `error_json()` · `app_error_handler()`

| 항목 | 내용 |
|---|---|
| 파일 | `studionx/backend/app/core/errors.py` |
| 작업 | P1-03 백엔드 공통 — 오류 응답 형식 |
| 작성 | Claude (AI) · 2026-10-06 · 사용자 검토 |
| 다음 | 02 validation error handler() (입력값 형식 오류) |

## 1. 하는 일

서버 코드가 `raise AppError(...)` 로 던진 오류를 정해진 모양의 JSON 응답으로 바꾼다.

```
raise AppError(ErrorCode.EMAIL_TAKEN, 409, "This email is already in use.")
```

→ 상태 코드 **409**, 본문:

```
{"error": {"code": "EMAIL_TAKEN", "message": "This email is already in use.", "details": null}}
```

| 함수 | 역할 |
|---|---|
| `error_json()` | 오류 응답 하나를 만든다. 앞으로 만들 처리기 02 ~ 04 도 모두 이 함수를 쓴다 |
| `app_error_handler()` | `AppError` 가 생기면 FastAPI 가 부르는 함수. `error_json()` 으로 응답을 만든다 |

## 2. 언제 쓰나

서버가 "요청은 받았지만 처리할 수 없다"고 알릴 때마다 쓰인다. 라우터 · 서비스 코드는 `raise AppError(...)` 한 줄만 쓰고, 응답 모양은 이 함수가 만든다.

| 상황 (구현 작업) | 던지는 오류 | 화면에서 일어나는 일 |
|---|---|---|
| 가입할 때 이미 쓰는 이메일 (P1-04) | `EMAIL_TAKEN` · 409 | 가입 화면의 이메일 칸 아래에 "This email is already in use." |
| 로그인 없이 마이페이지 API 호출 (P1-03 `get_current_user`) | `UNAUTHORIZED` · 401 | 프론트 `apiFetch` 가 토큰을 재발급한 뒤 다시 보냄 (JS 02) |
| 이메일 · 비밀번호가 틀림 (P1-05) | `INVALID_CREDENTIALS` · 401 | 로그인 모달에 오류 문구 |
| 다른 사람의 프로젝트를 열려고 함 (P1-13) | `FORBIDDEN` · 403 | 접근 거부 문구 |

### FastAPI 가 오류를 응답으로 바꾸는 방법

| 순서 | 일어나는 일 |
|---|---|
| ① 서버 시작 | `main.py` 가 `register_exception_handlers(app)` 를 부르고, 이 함수가 "`AppError` 는 `app_error_handler` 가 처리한다"고 등록한다 |
| ② 요청 처리 중 | 라우터 · 서비스 어디에서든 `raise AppError(...)` |

| 순서 | 일어나는 일 |
|---|---|
| ③ FastAPI 가 오류를 잡음 | 오류의 종류(`AppError`)로 등록된 처리기를 찾는다 |
| ④ 처리기 실행 | `app_error_handler(request, exc)` → JSON 응답 |
| ⑤ 브라우저 | `api.js` 가 `error.code` 를 읽어 `ApiError` 로 만든다 (JS 02) |

## 3. 코드

**import** (파일 맨 위)

```
from enum import StrEnum

from fastapi import FastAPI, Request
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse
from starlette.exceptions import HTTPException as StarletteHTTPException

from app.schemas.common import ErrorBody, ErrorResponse
```

`errors.py` 맨 위의 import 이다 (02 ~ 04 에서 쓰는 것까지 모두 있음). 이 함수들은 **`FastAPI`** · **`Request`** · **`JSONResponse`** · **`ErrorBody`** · **`ErrorResponse`** 를 쓴다. `AppError` · `ErrorCode` 는 같은 파일 안에 있다.

| import | 가져오는 것 | 쓰는 곳 |
|---|---|---|
| `StrEnum` (파이썬 기본) | 값이 글자인 열거형 | `ErrorCode` |
| `FastAPI` · `Request` | 앱 · 요청 타입 | `register_exception_handlers(app)` · 처리기의 `request` |
| `RequestValidationError` | 입력 형식 오류의 종류 | 02 |
| `JSONResponse` | JSON 응답 | `error_json()` |
| `StarletteHTTPException` | 404 · 405 같은 기본 오류의 종류 (Starlette 의 `HTTPException`) | 03 |
| `ErrorBody` · `ErrorResponse` | 오류 응답 모양 (Pydantic, `app/schemas/common.py`) | `error_json()` |

**함수**

```
def error_json(
    status_code: int,
    code: str,
    message: str,
    details: dict | None = None,
) -> JSONResponse:
    body = ErrorResponse(error=ErrorBody(code=code, message=message, details=details))
    return JSONResponse(status_code=status_code, content=body.model_dump())


async def app_error_handler(request: Request, exc: AppError) -> JSONResponse:
    return error_json(exc.status_code, exc.code, exc.message, exc.details)
```

```
def register_exception_handlers(app: FastAPI) -> None:
    app.add_exception_handler(AppError, app_error_handler)
    # TODO: RequestValidationError · StarletteHTTPException · Exception 처리기 등록
```

### 문법 풀이

| import 문법 | 뜻 |
|---|---|
| `from 모듈 import 이름` | 모듈 안의 `이름` 만 가져와 그대로 씀 (예: `Request`) |
| `app.…` | 우리 프로젝트(`studionx/backend/app/`)의 파일. `app.core.errors` = `app/core/errors.py` |

| 식 | 뜻 |
|---|---|
| `status_code: int` | **타입 힌트**: 이 값은 정수라는 표시. 실행에는 영향이 없고 편집기 · 검사 도구가 쓴다 |
| `dict \| None = None` | 사전(dict) 또는 `None`. 넘기지 않으면 `None` |
| `-> JSONResponse` | 이 함수가 돌려주는 값의 종류 |
| `ErrorResponse(error=ErrorBody(...))` | `schemas/common.py` 의 Pydantic 모델. 정해 둔 칸(`code` · `message` · `details`)만 받는다 |
| `body.model_dump()` | Pydantic 모델 → 파이썬 사전. `JSONResponse` 가 이 사전을 JSON 글자로 바꾼다 |
| `async def` | 비동기 함수. FastAPI 의 오류 처리기는 이 형태로 만든다 |
| `request: Request` | 오류가 난 요청. 지금은 쓰지 않지만 FastAPI 가 항상 넘겨준다 (P1-22 에서 API · 페이지 요청을 나눌 때 씀) |
| `app.add_exception_handler(AppError, app_error_handler)` | "`AppError` 가 생기면 이 함수를 불러라"라고 등록 |

| `raise AppError(...)` | `status_code` | 본문 `error` |
|---|---|---|
| `(EMAIL_TAKEN, 409, "This email is already in use.")` | 409 | `{"code": "EMAIL_TAKEN", "message": "This email is already in use.", "details": null}` |
| `(VALIDATION_ERROR, 422, "Check the password requirements.", details={"password": ["upper", "special"]})` | 422 | `details` 에 `{"password": ["upper", "special"]}` |

## 4. 흐름

1. 서버 시작 때 `AppError` → `app_error_handler` 를 등록한다.
2. 요청 중 `AppError` 가 생기면 FastAPI 가 `app_error_handler` 를 부른다.
3. `error_json()` 이 `{"error": {...}}` 응답을 오류에 적힌 상태 코드로 만든다.

## 5. 왜 이렇게

| 선택 | 이유 |
|---|---|
| 응답을 만드는 함수(`error_json`)를 따로 둠 | 처리기가 4개(이 함수 + 02 ~ 04)가 된다. 모양을 한 곳에서만 만들어야 어긋나지 않는다 |

| 선택 | 이유 |
| --- | --- |
| Pydantic 모델(`ErrorResponse`)을 거쳐 만듦 | 칸 이름을 잘못 쓰면 바로 오류가 난다. API 문서(`/docs`)에 나오는 모양과 실제 응답이 같다 |
| 처리기는 `AppError` 의 값을 그대로 씀 | 상태 코드 · 문구는 오류를 던지는 곳이 가장 잘 안다. 처리기는 모양만 맞춘다 |
| `@app.exception_handler` 대신 `add_exception_handler` | 처리기를 보통 함수로 두면 시험 · 문서에서 이름으로 가리킬 수 있다. 동작은 같다 |
| `message` 는 영어 | 화면이 영어다. 화면 번역은 프론트가 `code` 로 한다 |

## 6. 시험 (`tests/test_errors.py`)

시험용 작은 FastAPI 앱에 처리기만 등록하고, 일부러 `AppError` 를 던지는 주소 3개로 확인했다.

| 경우 | 결과 |
| --- | --- |
| `EMAIL_TAKEN` · 409 | 상태 409, `application/json`, 본문이 정확히 `{"error": {code, message, details: null}}` |
| `details` 가 있는 오류 (422) | `details` 가 그대로 전달됨 |
| `UNAUTHORIZED` · 401 | `code` 가 `"UNAUTHORIZED"` — 프론트가 재발급을 시도하는 조건과 같음 |

전체 시험 17개 통과, `ruff` 검사 통과.